

The MCP Guide

Connecting Model Context Protocol servers to LevelCode — the security model, a working GitHub integration, and the image-generation servers worth your time.

VERSION

1.0

DATE

28 July 2026

APPLIES TO

LevelCode with MCP slices S1–S6a

TRANSPORT

stdio (HTTP is S6b)

STATUS OF §4

Installed & verified on this machine

REFERENCE

`docs/MCP.md` in the editor repo

Contents

PART I — FOUNDATIONS

- 1 What MCP is, and what LevelCode does with it
- 2 The security model

PART II — SETUP

- 3 Configuring a server
- 4 GitHub MCP — a worked example

PART III — IMAGE GENERATION

- 5 The rule that decides everything
- 6 The servers worth your time

- 7 Reaching hosted servers
- 8 Cost, autopilot, and the allow-list

PART IV — OPERATING

- 9 Day-to-day
- 10 Troubleshooting

APPENDICES

- A Settings reference
- B What was verified, and how

How to read this

Sections 1–4 are the same for every MCP server you will ever add; section 4 walks through a real one that is already installed and working on this machine. Sections 5–8 are image generation specifically, and section 5 is the one to read first — it eliminates roughly half the servers on the market for reasons that have nothing to do with image quality.

A NOTE ON CLAIMS

Everything marked **VERIFIED** was executed against LevelCode's own code on 28 July 2026, not recalled or inferred. Everything marked **PUBLISHED** comes from the vendor's own documentation and was not run here. Appendix B lists exactly which is which, so you know where to be sceptical.

1 What MCP is, and what LevelCode does with it

The Model Context Protocol is a standard way for a program to hand an AI agent a set of tools. A *server* publishes tools; a *client* — LevelCode — connects, reads the list, and offers them to the model alongside its own built-ins. That is the whole idea. Everything interesting is in the details of trust and cost.

1.1 What a server actually is

An MCP server entry is a **process to spawn**. Not a URL, not a plugin, not a sandboxed extension — a command line that LevelCode will execute on your machine with your permissions. `npx -y some-mcp-server` downloads and runs code from npm. This is the single fact that motivates the entire security model in section 2, and it is worth sitting with before you paste a config block from a blog post.

1.2 What LevelCode implements

LevelCode speaks a deliberately small slice of MCP. Knowing the boundary saves you from debugging things that were never going to work:

CAPABILITY	STATUS	NOTES
stdio transport	YES	Spawn a process, speak newline-delimited JSON-RPC 2.0 over its stdin/stdout.
Tools	YES	<code>tools/list</code> and <code>tools/call</code> , namespaced and policy-gated.
Streamable HTTP	NOT YET	Slice S6b. Hosted servers need a bridge today — see section 7.
Resources & prompts	NOT YET	A server may advertise them; LevelCode ignores them.
Sampling	NO	A server driving your model is a deliberate omission, not a gap.
Inline image results	DROPPED	The consequence that shapes all of Part III. See section 5.

1.3 What one tool call costs you

Two costs, and only one of them is obvious. The obvious one is the call itself — latency, and for a paid API, money. The quiet one is that **every tool schema rides every turn**. A server advertising sixty tools spends part of your context window on every message you send, whether or not you use it. LevelCode shows this: the context-usage popover carves out an *MCP tools* segment so the standing cost is visible rather than mysterious.

This is why the GitHub server in section 4 is configured with a `GITHUB_TOOLSETS` restriction rather than everything it can do. Treat the tool list as a budget.

2 The security model

Four gates, called G1–G4 in `docs/MCP.md`. They exist because "add an MCP server" is a request to run arbitrary code, and because the text a server returns is read by a model that will act on it.

2.1 Two config tiers, two trust levels

A server can be defined in your **user settings** or in a repo's `.levelcode/mcp.json`. These are not equally trusted, and LevelCode keeps track of which is which for the life of the entry.

TIER	AUTHORED BY	CONSEQUENCE
settings	You	Starts without asking. Wins any name collision — a repo can never redefine a server you defined.
workspace	Whoever wrote the repo	Attacker-controlled for any repo you clone. Cannot start until you approve the literal command line (G1).

The setting is declared `application`-scoped for exactly this reason: a workspace cannot override it. That is also why the **Add a server...** wizard writes only the global tier — offering the workspace tier would quietly undo the guarantee.

2.2 G1 — the launch gate

A repo-defined server does not start on clone-and-open. The first time the agent needs it, you get a consent card showing the **exact command, arguments, and environment** that would be spawned. Approving stores a SHA-256 fingerprint of that launch, scoped to this workspace.

WHY A FINGERPRINT, NOT A NAME

If trust were keyed on the server's name, a repo could get a harmless entry approved and then edit the command. Because the stored value is a hash of the whole launch, changing so much as one argument invalidates it. LevelCode reports that case distinctly — `command changed` — `needs approval` — rather than folding it in with "never approved", because the two mean very different things.

Approval is reversible: **Manage MCP servers...** → **Revoke trust**, per server or workspace-wide. Revoking does not kill a server already running in this session; it means the next run asks again.

2.3 G2 — per-call approval

Every MCP tool call defaults to **ask**. Not "ask for writes" — ask for everything. The only thing that grants silent execution is your explicit allow-list, `levelcode.ai.mcp.toolPolicy`:

```
// keys are the NAMESPACE name: server__tool
"levelcode.ai.mcp.toolPolicy": {
  "github__pull_request_read": "allow",    // silent
  "github__merge_pull_request": "ask"      // card every time (also the default)
}
```

"*": "allow" is accepted and is almost always a mistake — read section 8 before you reach for it. One rule outranks your allow-list: if a server marks a tool with `destructiveHint: true`, it always asks and the policy cannot override that.

2.4 G3 — autopilot is not a bypass

Autopilot lets the agent run its own built-in tools without stopping. It does **not** extend to third-party MCP tools. An MCP call that is not explicitly allow-listed still prompts, autopilot or not. This is the gate that keeps "let the agent work" from silently meaning "let any repo's server act on my behalf".

2.5 G4 — text from a server is untrusted input

Tool descriptions and tool results are third-party strings that a model reads and acts on. That is the classic prompt-injection surface, and there is no complete fix — LevelCode bounds descriptions (1,024 chars) and results (24,000 chars) and is explicit that bounding is mitigation, not a solution. Practical consequence: **be more careful allow-listing a tool whose output you haven't seen** than one you have.

2.6 The path of a single call

- 1** Merge config — settings first, then any workspace file, with provenance attached.
- G1** Repo-defined? Check the launch fingerprint. No match → consent card → nothing spawns until you say yes.
- 2** Spawn with `shell:false`, detached so the whole process group can be killed.
- 3** `initialize` → `tools/list`, 20 s budget for the whole handshake.
- 4** Namespace to `server__tool`, sanitise, truncate to 64 chars, never shadow a built-in.
- G2** Classify the call: `allow` only from your policy, otherwise the approval card.
- 5** Call, with a timeout. Flatten the result to text, capped at 24,000 chars.
- ✓** Result reaches the model as a string. Failures arrive as `ERROR: ...`, never as a thrown exception that could break the turn.

3 Configuring a server

3.1 Three ways, in order of preference

Manage MCP servers... EASIEST

Command Palette → **AI: Manage MCP Servers...**, or the **Manage servers...** link at the foot of `/mcp` in chat.

```
LevelCode: Manage MCP servers...
✓ github    your settings · running · 41 tools · 25 allow-listed
△ fs        .levelcode/mcp.json · command changed — needs approval
○ replicate your settings · not started

————— Actions —————

+ Add a server...
🔗 Open settings JSON
🔄 Revoke all workspace trust...
```

The wizard asks for a name, a command, and an argument line. The argument line is **quote-aware**, which matters more than it sounds: `-y server-filesystem "/Users/you/My Documents"` is one of the most common MCP configs in existence, and a naive whitespace split turns that path into three broken arguments. It is a splitter, not a shell — no `$VAR` expansion, no globbing — matching how the process is actually spawned.

settings.json FULL CONTROL

The wizard has no `env` step; when a server needs one, edit the file directly.

```
// ~/Library/Application Support/LevelCode/User/settings.json
"levelcode.ai.mcp.servers": {
  "replicate": {
    "command": "npx",
    "args": ["-y", "replicate-mcp"],
    "env": { "REPLICATE_API_TOKEN": "r8_..." }
  }
}
```

.levelcode/mcp.json REPO-AUTHORED

Same shape, committed to a repo so a team shares one config. Everything in it is **untrusted** and gated by G1. Both `{ "mcpServers": {...} }` and a bare `{ name: {...} }` map are accepted, so configs from other clients paste straight in.

3.2 Secrets: the part most guides get wrong

DO NOT PUT A LIVE API TOKEN IN SETTINGS.JSON

It is plaintext on disk, it is included in Settings Sync, and it lands in any screenshot of your settings file. Every vendor README shows this pattern because it is the shortest thing to document — not because it is the right thing to do.

Two better options, in order:

1. **A wrapper script that fetches the secret at spawn time** from a keychain or credential helper. The token never exists in a file. Section 4.2 is a complete worked example.
2. **Inherit from the environment.** LevelCode spawns servers with `process.env` inherited, so exporting the variable in your shell works — with one caveat: *macOS GUI apps do not inherit your shell environment*. This only works if you launch LevelCode from a terminal. The wrapper approach works either way, which is why it is first.

A CONFIG THAT WORKS EVERYWHERE ELSE AND FAILS HERE

Several READMEs — Replicate Flux's among them — show this for Cursor:

```
"command": "env REPLICATE_API_TOKEN=YOUR_TOKEN npx",  
"args": ["-y", "replicate-flux-mcp"]
```

LevelCode spawns with `shell:false`, so that entire string is looked up as *one executable filename*. You get `ENOENT`, not a shell command. Use the `env` block form instead — or better, section 4.2.

4 GitHub MCP — a worked example

INSTALLED ON THIS MACHINE — VERIFIED 28 JULY 2026

Connected through LevelCode's own client: handshake 1.8 s, 41 tools, every name legal, a live pull request read back, and write access confirmed by probe. 25 read tools allow-listed; all 16 write tools still gated.

4.1 Why a wrapper and not the standard config

GitHub's hosted MCP server at `api.githubcopilot.com/mcp/` is Streamable HTTP, which LevelCode does not speak yet. The same server ships as a container that speaks stdio, which is a clean fit — and running it locally means the token problem is ours to solve properly. The wrapper below pulls the token from the `gh` CLI's keychain entry at spawn time, so it never exists in a file and rotating it needs no config change.

4.2 The wrapper

~/LEVELCODE/MCP/GITHUB-MCP · CHMOD 755

```
#!/bin/sh
set -eu

GH=${GH_BIN:-/opt/homebrew/bin/gh}
DOCKER=${DOCKER_BIN:-/usr/local/bin/docker}

# Every tool schema rides EVERY turn, so this is a context budget, not a preference.
: "${GITHUB_TOOLSETS:=pull_requests,repos,issues,context}"
export GITHUB_TOOLSETS

GITHUB_PERSONAL_ACCESS_TOKEN=$(("$GH" auth token 2>/dev/null) || GITHUB_PERSONAL_ACCESS_TOKEN='')
if [ -z "$GITHUB_PERSONAL_ACCESS_TOKEN" ]; then
    echo 'github-mcp: no GitHub token – run `gh auth login`' >&2
    exit 78
fi
export GITHUB_PERSONAL_ACCESS_TOKEN

# -i keeps stdin open: that IS the JSON-RPC channel. --rm leaves no dead containers.
exec "$DOCKER" run -i --rm \
    -e GITHUB_PERSONAL_ACCESS_TOKEN \
    -e GITHUB_TOOLSETS \
    ghcr.io/github/github-mcp-server
```

The settings entry is then trivial, and contains no secret at all:

```
"levelcode.ai.mcp.servers": {
  "github": { "command": "/Users/you/.levelcode/mcp/github-mcp", "args": [] }
}
```


TRADE-OFF, STATED PLAINLY

The token lives in the wrapper process's environment, so `ps -E` can reveal it — but only to the same user, who can simply run `gh auth token`. That is a real weakening of nothing. Compare it with a plaintext credential in a synced settings file, which survives backups, screen shares, and support tickets.

4.3 What you can do with it

ASK	TOOL
Read a PR — diff, files, reviews, comments, checks	<code>github__pull_request_read</code> (9 <code>method</code> values)
List / search PRs	<code>github__list_pull_requests</code> , <code>github__search_pull_requests</code>
Create a PR	<code>github__create_pull_request</code>
Update title, body, base, reviewers, draft	<code>github__update_pull_request</code>
Close a PR	<code>github__update_pull_request</code> with <code>state: "closed"</code>
Review — approve, request changes, comment	<code>github__pull_request_review_write</code>
Merge	<code>github__merge_pull_request</code>
Files, commits, branches, issues, code search	25 further tools across the enabled toolsets

There is no separate "close PR" tool — closing is an update with `state: "closed"`. Worth knowing, because asking the agent to "close PR 42" works, but only because the model maps it onto `update_pull_request`.

4.4 The allow-list as installed

Reads run silently; anything that changes state on GitHub shows a card. This is the line that matters, because autopilot is enabled in this profile.

POLICY	COUNT	EXAMPLES
ALLOW reads	25	<code>pull_request_read</code> , <code>list_pull_requests</code> , <code>get_file_contents</code> , <code>search_code</code> , <code>list_commits</code> , <code>issue_read</code> , <code>get_me</code> ...
ASK writes	16	<code>create_pull_request</code> , <code>update_pull_request</code> , <code>merge_pull_request</code> , <code>push_files</code> , <code>create_or_update_file</code> , <code>delete_file</code> , <code>create_branch</code> , <code>fork_repository</code> , <code>issue_write</code> , <code>pull_request_review_write</code> , <code>add_issue_comment</code> ...

NO GITHUB TOOL SETS DESTRUCTIVEHINT

Which means none of them trip G2's hard force-ask rule — the allow-list *can* grant every one of them, including `merge_pull_request` and `delete_file`. A `"*": "allow"` here would let an autopilot run merge a pull request without asking. The 25/16 split above is deliberate.

4.5 Things to try

- “Read PR 58 in levelcodeai/levelcode and summarise what the reviewer would push back on.”
- “What changed in the last five commits on `develop` ? Group by subsystem.”
- “Open a PR from this branch to `develop` , and write the body from the commit messages.” → approval card
- “Close PR 41, it's superseded by 58.” → approval card
- “Find every place in this org's repos that still calls `oauth_start` .”

4.6 Adding capabilities

`GITHUB_TOOLSETS` in the wrapper controls the surface. Add `actions` for CI runs and logs, `notifications` , `discussions` , `gists` , `security` , or `all` . Each addition costs context on every turn forever — check `/mcp` and the context popover after changing it, and be honest about whether you actually use it.

5 The rule that decides everything

INLINE IMAGE RESULTS ARE DISCARDED

A tool result reaches the agent as a **string**. When a server returns an MCP `image` content block — base64 bytes inline, the obvious way to return a picture — LevelCode replaces it with a placeholder. The model never sees the image, and neither do you.

Measured against `flattenContent` on 28 July 2026 VERIFIED:

WHAT THE SERVER RETURNS		WHAT THE AGENT RECEIVES
<code>image</code> block, base64	LOST	<code>[image image/png omitted – the transcript is text]</code>
<code>text</code> block with a URL	WORKS	<code>https://replicate.delivery/xyz/out-0.webp</code>
<code>structuredContent</code> , no text block	WORKS	<code>{"url":"https://fal.media/files/a/b.png","seed":42}</code>
Writes a file, reports the path	WORKS	<code>Saved to /Users/you/Pictures/gen/out.png</code>
Text <i>and</i> an image block	PARTIAL	Text survives, image dropped — usually enough, if the text carries the URL

THEREFORE

Choose a server that returns a URL or writes to disk. This single criterion matters more than model quality, price, or speed, because a server that only returns inline bytes produces nothing usable here at any price. It is also the criterion no vendor comparison will mention, because for most clients it does not apply.

Two practical corollaries. First, a server with a configurable output directory — Recraft's `IMAGE_STORAGE_DIRECTORY` , for instance — is strictly better than one without, because the file lands somewhere you can open and the agent can edit around. Second, the 24,000-character result cap makes base64 unworkable regardless: a single 512×512 PNG blows past it.

6 The servers worth your time

Package data below was read from the npm registry on 28 July 2026; tool names come from each project's own documentation. Every configuration block in this section was run through LevelCode's `loadServerConfig` and tool-naming layer and accepted VERIFIED — but the servers themselves were not executed here.

SERVER	WEEKLY INSTALLS	RETURNS	KEY	BEST FOR
Replicate official <code>replicate-mcp</code>	1,554	URL	<code>REPLICATE_API_TOKEN</code>	The default choice. Whole Replicate catalogue, one server.
fal.ai <code>fal-ai-mcp-server</code>	459	URL	<code>FAL_KEY</code>	Speed. Best raw throughput on FLUX; 600+ models.
Recraft <code>@recraft-ai/mcp-recraft-server</code>	233	URL + file	<code>RECREFT_API_KEY</code>	Vectors. True SVG output, plus upscale and background tools.
Replicate Flux <code>replicate-flux-mcp</code>	141	URL	<code>REPLICATE_API_TOKEN</code>	A friendlier <code>generate_image</code> over the same API.
EverArt reference <code>@modelcontextprotocol/server-everart</code>	175	URL	<code>EVERART_API_KEY</code>	Minimal. One tool. Good for a first taste of the flow.
ComfyUI community	—	varies	NONE	Local, free, private. Check the result shape first.

6.1 Replicate — start here

The official server, maintained by Replicate, covering their whole HTTP API rather than one model. You search for a model and create a prediction, which means new models work the day they land instead of waiting for a server update.

```
"replicate": {
  "command": "npx",
  "args": ["-y", "replicate-mcp"],
  "env": { "REPLICATE_API_TOKEN": "r8_..." }
}
```

DOTTED TOOL NAMES GET REWRITTEN

Replicate's tools are named `models.search`, `predictions.create` and so on. A dot is illegal in a tool name, so LevelCode sanitises it — and your allow-list must use the *sanitised* name or it will silently never match:

```
models.search      → replicate__models_search
predictions.create → replicate__predictions_create

"levelcode.ai.mcp.toolPolicy": {
  "replicate__models_search": "allow",    // searching is free
  "replicate__predictions_create": "ask"  // creating costs money
}
```

Run `/mcp` once after connecting: it prints the real namespaced names, which is the only reliable way to write the policy.

6.2 fal.ai — when speed matters

Serverless-first and built for low latency; independent comparisons in 2026 put it ahead of Replicate on raw FLUX throughput and meaningfully cheaper, with the trade-off that Replicate has the broader catalogue and better documentation. Replicate also suffers cold starts of 20–60 seconds on idle models — worth knowing against LevelCode's 20-second handshake budget and per-call timeout.

```
"fal": {
  "command": "npx",
  "args": ["-y", "fal-ai-mcp-server"],
  "env": { "FAL_KEY": "..." }
}
```

6.3 Recraft — the one that does vectors

The interesting outlier. Recraft generates **true SVG**, not a raster traced into paths, which makes it the only option here that produces an asset you can commit, diff, and restyle with CSS. For icons, logos, and diagram assets in a codebase, that is a categorical difference rather than a quality difference. It also exposes vectorise, background removal and replacement, and two upscalers.

```
"recraft": {
  "command": "npx",
  "args": ["-y", "@recraft-ai/mcp-recraft-server@latest"],
  "env": {
    "RECRRAFT_API_KEY": "...",
    "IMAGE_STORAGE_DIRECTORY": "/Users/you/Pictures/recraft"
  }
}
```

Set `IMAGE_STORAGE_DIRECTORY`. It makes the server write files and report paths, which is exactly the shape section 5 says to prefer.

THE NPM PACKAGE IS DEPRECATED

Its own README now points at a hosted server, <https://mcp.recraft.ai/mcp>. The npm build still works and was last published 13 July 2026, but treat it as a bridge — section 7 shows how to reach the hosted one, which is where the maintenance is going.

6.4 ComfyUI — local, free, private

If you already run ComfyUI, a community MCP server puts your existing workflows behind a tool call: no API key, no per-image cost, nothing leaving the machine. Several implementations exist and they vary in quality — the one thing to check before committing is **how it returns an image**. Many return inline base64, which section 5 rules out. Prefer one that saves to a directory and reports the path.

```
// shape only — pick a specific implementation and read its README first
"comfy": {
  "command": "uvx",
  "args": ["comfyui-mcp-server"],
  "env": { "COMFYUI_URL": "http://127.0.0.1:8188" }
}
```

VET COMMUNITY SERVERS BEFORE YOU ADD ONE

A server entry is a command to execute. A single-maintainer package with a hundred weekly installs is a supply-chain decision, not a preference — read the source, pin the version rather than using `@latest`, and prefer projects with a real commit history. This applies to every server in this section except the official Replicate and reference EverArt builds.

7 Reaching hosted servers

Several vendors now recommend a hosted endpoint over a local package — Replicate (`mcp.replicate.com`), Recraft (`mcp.recraft.ai`), GitHub (`api.githubcopilot.com/mcp/`). These speak Streamable HTTP, which LevelCode will support in S6b. Until then, one bridge covers all of them.

`mcp-remote` is a stdio server that proxies to a remote one and handles the OAuth dance. At roughly 511,000 weekly installs it is the de-facto standard for exactly this gap, and it is the same mechanism other stdio-only clients use.

```
"replicate-remote": {
  "command": "npx",
  "args": ["-y", "mcp-remote", "https://mcp.replicate.com/mcp"]
}
```

Weigh it honestly:

FOR	AGAINST
• Browser OAuth — no API token anywhere on disk • Always the vendor's current version • Works today, no waiting for S6b	• A second process in the chain to debug • First run opens a browser — surprising inside an editor • Another dependency between you and the API

Rule of thumb: if the vendor ships a maintained local package, use it. Reach for the bridge when the local package is deprecated (Recraft) or was never offered.

8 Cost, autopilot, and the allow-list

IMAGE TOOLS SPEND MONEY — AND AUTOPILOT IS ON IN THIS PROFILE

Allow-listing `generate_image` means the agent can call it without asking, as many times as it decides to. An agent iterating on a prompt can produce twenty images in a minute. For a GitHub read tool the cost of over-permissioning is a wasted API call; here it is a bill.

The split that works in practice:

POLICY	TOOLS	WHY
ALLOW	search and list models, get a model schema, fetch a prediction's status or result, read the account	Free, read-only, and the agent needs them constantly to pick a model.
ASK	anything that generates, upscales, vectorises, or edits	Each call is a charge. The card is also where you catch a runaway loop.

```
"levelcode.ai.mcp.toolPolicy": {
  // free – let the agent explore
  "replicate__models_search":    "allow",
  "replicate__models_get":      "allow",
  "replicate__predictions_get": "allow",
  // costs money – always show the card
  "replicate__predictions_create": "ask",
  "recraft__generate_image":      "ask",
  "recraft__creative_upscale":    "ask"
}
```

A WORKING IMAGE SETUP, END TO END

1. Add **Replicate** for range and **Recraft** for vectors. Two servers is plenty; each one costs context on every turn.
2. Set `IMAGE_STORAGE_DIRECTORY` on Recraft so results land as files.
3. Run `/mcp` and copy the real namespaced tool names.
4. Allow-list the free read tools only. Leave every generate on `ask`.
5. Check the context popover. If the MCP segment is uncomfortable, drop a server rather than living with it.

9 Day-to-day

9.1 `/mcp`

Type `/mcp` in chat. It lists **configured** servers, not running ones — deliberately, because the questions it exists to answer are "why isn't my server being used?" and "what is this repo asking to run?", and a list of live processes answers neither. Each row shows state, provenance, the literal command line, and — when connected — the real tool names with their allow-list state. Those names come from the same code path the agent uses, so the list cannot claim a tool is allowed while the agent refuses it.

9.2 Watching the context cost

The context-usage popover has an **MCP tools** segment, carved out of the existing Tools slice rather than added next to it, so the bar still sums correctly. It appears only when a server contributed a tool. Check it after adding a server; it is the standing rent you pay every turn.

9.3 Managing trust

Manage MCP servers... → pick a repo-defined server → **Revoke trust for this workspace**. Or revoke everything at once from the Actions section. If a row says `command changed – needs approval`, read the new command line before re-approving: that message means the repo edited what it wants to run *after* you approved it.

10 Troubleshooting

SYMPTOM	CAUSE	FIX
Server never starts, no error	Repo-defined, awaiting G1	<code>/mcp</code> shows <code>needs approval</code> . Ask the agent to use it; the consent card appears.
<code>ENOENT</code> on a command that works in your terminal	<code>shell: false</code> , or a GUI launch with a different <code>PATH</code>	Use an absolute path. Never <code>"command": "env F00=1 npx"</code> .
Server starts from a terminal launch, not from the Dock	macOS GUI apps don't inherit your shell env	Use a wrapper script (§4.2) instead of relying on exported variables.
Allow-list entry ignored	Wrong key — raw name, or a sanitised one you guessed	Copy the exact namespaced name from <code>/mcp</code> . Dots become underscores.
Agent says the image was "omitted"	Server returned inline base64	Nothing to configure — switch to a server that returns a URL or a path (§5).
Result ends in <code>...[MCP result truncated]</code>	Over 24,000 chars	Ask for a narrower query; the cap protects the context window.
Handshake times out	Cold start over the 20 s budget	Common on idle Replicate models. Pre-pull images; retry once warm.
Duplicate-server warning	Same name in settings and a repo file	Working as designed — yours wins. Rename the repo's entry if you want both.
Tool asks every time despite <code>allow</code>	Server sets <code>destructiveHint: true</code>	Not overridable. That is the point.
Docker server fails after a restart	Docker Desktop not running	Start Docker. The wrapper's stderr surfaces in the <code>ERROR: string</code> .

A Settings reference

KEY	SCOPE	PURPOSE
levelcode.ai.mcp.servers	application	Name → { command, args, env }. Application-scoped so a workspace cannot override it.
levelcode.ai.mcp.toolPolicy	application	Namespaced tool name → "allow" "ask" . "*" matches everything.
.levelcode/mcp.json	repo	Per-workspace servers. Untrusted; gated by G1.

Limits

BOUND	VALUE	WHY
Servers	20	A config with 500 is a mistake, not a use case
Tools per server	64	Every schema rides every turn
Tool-name length	64	Stricter of Anthropic's 128 and OpenAI's 64
Tool description	1,024	Context cost, and the G4 injection surface
Result	24,000	Truncated, not dropped
Handshake	20 s	spawn + initialize + tools/list

Commands

AI: Manage MCP Servers...	Add, remove, inspect, revoke trust
/mcp	In chat — configured servers, state, tools, allow-list

B What was verified, and how

So you know which claims in this document rest on a measurement and which rest on someone else's documentation.

Executed on this machine — 28 July 2026

CLAIM	EVIDENCE
GitHub MCP connects through LevelCode's client	Container spawned via <code>mcpClient.connect</code> ; handshake 122 ms direct, 1,827 ms through the keychain wrapper
41 tools, all names legal under the 64-char rule	<code>buildAgentTools</code> , then every name matched against <code>^[A-Za-z0-9_-]{1,64}\$</code>
Reads and writes both work	PR #93 and PR #58 read back live; write access probed with an empty <code>update_pull_request</code> — GitHub authorised it, then rejected the empty body. Nothing was modified.
The installed policy is 25 allow / 16 ask	Real <code>settings.json</code> parsed and every tool classified through <code>classifyMcpTool</code> ; zero write tools allowed
Inline images are dropped; URLs and paths survive	Five result shapes through <code>flattenContent</code> — the table in §5
Every config block in §6 and §7 is accepted	Each run through <code>loadServerConfig</code> ; tool names through the namespacing layer; none arrived pre-allowed
Replicate's dotted names are rewritten	Observed: <code>models.search</code> → <code>replicate__models_search</code>
Quoted paths survive the Add wizard	<code>parseArgv</code> on <code>--output "/Users/.../My Images/gen"</code> → one argument

Read from published sources, not executed here

Package versions and install counts	npm registry and downloads API, 28 July 2026
Image-server tool names	Each project's own README or vendor docs
Recraft's deprecation notice	The package's own README
fal.ai vs Replicate speed and pricing	Third-party 2026 comparisons; treat as directional
ComfyUI server behaviour	Implementations vary — verify the result shape yourself

NOT VERIFIED

No image-generation server was executed on this machine. Doing so means running third-party code and, for the hosted ones, spending money on an account. Their *configuration* was validated against LevelCode's rules; their *behaviour* is the vendor's claim. Section 5 gives you the one test that matters — check what a server returns before you build on it.